Research on Two-dimensional Cutting Problem with Defects

Keqiang Wu, Xiaoping Min and Defu Zhang
Department of Computer Science
Xiamen University
Xiamen, China
denmushi66@gmail.com

Abstract—**The packing problem is an optimization problem often encountered in industrial production. It plays a vital role in improving work efficiency, reducing waste, increasing revenue, and promoting industrial automation. This paper studies the sub-problem of the packing problem -- the two-dimensional cutting problem with defects. Based on the priority heuristic algorithm, we propose the CIP algorithm and the CATS algorithm for the processing of defects. The CIP algorithm adopts the defect collision processing strategy, and the CATS algorithm adopts the strategy of removing defects in advance. The experimental results of the two algorithms have their own advantages. But overall, the CIP algorithm performs better in most cases, and is much simpler from a data structure perspective. Our research results are of great value in practical applications.**

Packing problem; CIP algorithm; CATS algorithm; Heuristic algorithm

I. INTRODUCTION

The packing problem can be mainly divided into three-dimensional packing and two-dimensional packing. The three-dimensional packing problem refers to loading a specified three-dimensional cargo into a three-dimensional cubic container, ensuring that the whole cargo is completely placed in the container and that the goods are not overlapped, and the filling result satisfies a specific objective function under this condition. Compared with the three-dimensional packing problem, the two-dimensional packing is less complicated and the research starts earlier. It is widely used in the field of industrial cutting and has important theoretical value and practical application significance. The two-dimensional packing problem refers to loading a specified two-dimensional object in a two-dimensional container according to requirements, and ensuring that the objects do not overlap, and the goal is to reduce waste. Since the application scenario of the two-dimensional boxing problem is basically equivalent to cutting, therefore, we call it cutting problem here. According to the different objectives, the two-dimensional packing problem is mainly divided into three categories [1].

- Two-Dimensional Bin Packing Problem (2BP): The length and width of the container are fixed, and the goal is to completely pack the specified objects, minimizing the number of containers used.
- Two-Dimensional Strip Packing Problem (2SP): The width of the container is fixed and the length is not fixed. The goal is to load all objects and minimize the height of the container used.
- Two-Dimensional Knapsack Packing Problem (2KP): The length and width of the container are fixed, and the object has a corresponding value. The goal is to select a subset of objects to be loaded into the container to maximize the total value of the loaded object. If the value of the object corresponds to the area of the object, the target corresponds to maximizing the filling rate and minimizing the wasted area.

There are several assumptions in the study of two-dimensional packing problems:

1) whether the object is a rectangle.
2) whether the length of the container is fixed.
3) whether the object can be flipped at 90°.
4) whether it satisfies guillotine constraint.

Most studies satisfy the condition (1), which stipulates that the object is rectangular and only considers the orthogonal packing problem, that is, the edge of the object is parallel to the container. There are also a small number of studies on the cutting of irregular objects [2] [3], which basically limits the two conditions (3) and (4). We usually use R for rotation, O for no rotation, G for guillotine constraint, and F for not satisfying guillotine constraint. In this paper, we study the two-dimensional cutting problem with defects.

II. RELATED WORK

Research on the two-dimensional packing problem mainly focuses on accurate algorithms and heuristic algorithms. Accurate algorithms ensure that the optimal solution to the problem is found, but it is time consuming and inefficient. One of the earliest precise algorithms was solved by integer linear programming [4] [5]. Although the heuristic algorithm cannot guarantee the optimal solution, it is popular among scholars because of its superiority in time and the availability of results. First, scholars consider constructing a reasonable order of loading. Baker [6] proposed the Bottomleft priority algorithm in 1980. Chazelle [7] improved this algorithm.

978-1-7281-0945-9/19/$31.00©2019 IEEE

Authorized licensed use limited to: Universita degli Studi di Bologna. Downloaded on March 25,2026 at 14:29:37 UTC from IEEE Xplore. Restrictions apply.

Lodi [8] proposed a floor-ceiling algorithm and a touching perimeter algorithm. In addition, scholars are actively using artificial intelligence algorithms to explore the loading sequence. These strategies include tree search algorithm [4], simulated annealing algorithm [9] [10], genetic algorithm [11] [12] [13], artificial neural network algorithm [13], tabu search algorithm [8] [14], Iterative maximal area algorithm [15], iterative doubling binary search algorithm [16], the result of some algorithms is close to the optimal solution in some data sets, but the disadvantage is that it takes a lot of time to test on huge instance. The more effective algorithm is based on a certain filling strategy. Leung [17] proposed a fast heuristic algorithm based on scoring strategy. Zhang [18] proposed a binary search algorithm. The basic strategy is to ensure the smoothness of unused space. Two papers by Özcan [19] [20] can solve this problem in a shorter time. Wang [15] used a strategy of maximizing residual space to improve the experimental results. The hybrid heuristic search algorithm based on simulated annealing algorithm and binary search algorithm used by Hong [10] has achieved excellent filling results.

In 1968, Hahn [21] defined a problem of cutting stocks with defects and proposed a three-stage solution using dynamic programming algorithms. In 1988, Scheithauer and Terno [22] also used the dynamic programming algorithm to solve the problem of cutting a rectangular piece from a defective rectangular block with guillotine constraint. Inspired by the problem of cutting fabrics on roller blinds, De Gelder and Wagelmans [23] proposed a two-stage heuristic algorithm that selects the cutting mode in the first stage, and determine the order in which the pattern is to be cut in the second stage. A few scholars have studied the two-dimensional cutting problem of a single defect area [24] [25]. Afsharian, Niknejad, and Wäscher [26] used a heuristic algorithm based on dynamic programming. In their study, multiple defects could be allowed on a large object, but the number of small items was not limited. Wenshu, Dan, and Jinzhuo [27] proposed the use of genetic algorithms. They converge the optimal results by selecting, intersecting, and mutating, improving the utilization of defective wood. Regarding the online cutting of defects, Ghodsi and Sassani [28] define cutting stock problems with variable stock sizes from the solid wood furniture industry, where each stock (wood strip) has defects to be removed. In 2016, Tuzun Aksu and Durak [29] proposed a dynamic programming algorithm to solve the problem of online glass cutting. Glass products can contain different types and different number of defects. In 2017, they proposed a mixed integer programming formula to improve their algorithms.

From the above literature, we can see that the dynamic programming is often used to solve the two-dimensional packing problem with defects. The types of defects are block or dot, and the packing types are 2BP and 2SP. In this paper, the research of this problem is inspired by the cloth cutting. In the garment industry, large fabrics need to be cut into the specified shape and then sewed by tailors. However, the fabric is inevitably smashed, and the smashing place cannot be used for garments. This disrupts the normal cutting schedule, so the factory must find ways to minimize waste while maintaining quality and to cut more specified styles on the specified fabric. The defect shape is a rectangle, that is, there is a rectangular unavailable area in the middle of the container, and the object to be cut needs to bypass the area, and the goal is to maximize the container utilization. The research in this paper is similar to the specific problems studied by Wenshu, Dan, and Jinzhuo [27] above, but the number of experimental examples in their paper is small, and the number of defects is unique. This paper generates and expands the experimental use cases. We propose and compare three heuristic algorithms to solve this problem, and the evaluation criteria is the utilization rate.

III. Mathematical Model

Given a rectangular block, the length and width of the rectangular block are denoted as H and W, respectively, and the lower left corner of the rectangular block is set to the coordinate origin (0,0). There are irregularly distributed n rectangular defects on the rectangular block. The position of the defect d_i on the rectangular block is represented by the lower left corner coordinate $(x_1^{d_i}, y_1^{d_i})$ and the upper right coordinate $(x_2^{d_i}, y_2^{d_i})$ of the rectangle, where $0 < x_1^{d_i} < x_2^{d_i} \le W$, $0 < y_1^{d_i} < y_2^{d_i} \le H$, $0 < i \le n$. For example, $(0,0) \sim (2,3)$ represents a defect with a length and width of 3 and 2 in the lower left corner. There are m rectangular blocks to be cut, and the length and width of the rectangular block r_j are represented as h^{r_j} and w^{r_j}, respectively, where $0 < w^{r_j} \le W, 0 < h^{r_j} \le H$, $0 < j \le m$. We need to cut the rectangular block from the rectangular block to ensure that there is no overlap between the rectangular blocks and the defects. The goal is to maximize utilization. The position at which the rectangular block is cut is also represented by two coordinates $(x_1^{r_j}, y_1^{r_j})$ and $(x_2^{r_j}, y_2^{r_j})$, where $0 < x_1^{r_j} < x_2^{r_j} \le W$, $0 < y_1^{r_j} < y_2^{r_j} \le H$, $x_2^{r_j} - x_1^{r_j} = w^{r_j}, y_2^{r_j} - y_1^{r_j} = h^{r_j}$.

We define the binary variable θ_j, which takes a value of 1 when the rectangular block r_j is cut, otherwise 0. Define the binary variable β_{ij}, which takes a value of 1 when the defect and the rectangular block overlap, otherwise 0.

$$\beta_{ij} = \begin{matrix} 0 \; if \; x_2^{r_j} \le x_1^{d_i} || x_1^{r_j} \ge x_2^{d_i} || y_2^{r_j} \le y_1^{d_i} || y_1^{r_j} \ge y_2^{d_i} \\ x \; else \end{matrix} \quad (1)$$

So, the linear model of this problem can be expressed as:

$$Max \frac{\sum_{j=1}^{m} \theta_j * h^{r_j} * w^{r_j}}{H * W} \quad (2)$$

s.t.

$$\sum_{j=1}^{m} \theta_j * h^{r_j} * w^{r_j} \le H * W - \sum_{i=0}^{n} h^{d_i} * w^{d_i}, (i = 1,2,\ldots,u) \quad (3)$$

$$\beta_{ij} = 0, (i = 1,2,\ldots,m; j = 1,2,\ldots,n) \quad (4)$$

$$h^{d_i} = y_2^{d_i} - y_1^{d_i}, (i = 1,2,\ldots,n) \quad (5)$$

$$w^{d_i} = x_2^{d_i} - x_1^{d_i}, (i = 1,2,\ldots,n) \quad (6)$$

Authorized licensed use limited to: Universita degli Studi di Bologna. Downloaded on March 25,2026 at 14:29:37 UTC from IEEE Xplore. Restrictions apply.

The equation (2) indicates that the target is to maximize the utilization of the rectangular block, and the (3) indicates that the area of the cut rectangular block cannot be larger than the area after the rectangular block minus all the defects, and the (4) indicates that all the cut rectangular blocks cannot overlap with defects. The equation (5)(6) gives a representation of the length and width of the defect. This problem is actually a 2KP problem that maximizes the area of the rectangular block.

IV. Algorithm Description

This problem is different from the general two-dimensional rectangular block cutting, we need to deal with the defects. If the defect is removed or the defect is bypassed, the cutting can be smoothly performed. We apply the heuristic search tree algorithm described by Zhu [30] to solve this problem. Before the generation of the cutting strategy, the collision detection of the defective block is performed. If the collision is detected, the strategy is abandoned, otherwise the strategy is added to candidate set. Inspired by this idea, we propose two other algorithms to solve this problem. The difference between the two algorithms is how to deal with the defects. The first algorithm is called the CIP (cut defects in process) algorithm, which means that when the cutting is performed according to certain criteria, the defect collision detection is performed. If the defect occupies the available space and blocks the cutting process, we first consider changing the smaller rectangular block for cutting. If the remaining rectangular blocks are all inconsistent, we will cut the defect together with the unavailable space. The resection process is usually performed by cutting off the normal rectangular block. After the resection is completed, the cutting is continued according to the original cutting method until no space is available or all the rectangular blocks are cut. The second algorithm is called the CATS (cut defects at the start) algorithm. The algorithm first cuts the defects from the rectangular block to get all the remaining free space, and then cuts the remaining space in accordance with certain rules until there is no available space or all the rectangular blocks are cut. The cutting principles in both cutting algorithms use the priority heuristic algorithm proposed by Zhang [31] in a paper published in 2016, which is improved and applied to this problem. So, let's first introduce Zhang's priority heuristic algorithm, then introduce CIP algorithm and CATS algorithm separately.

A. *Priority heuristic algorithm*

Under guillotine constraint, recursive algorithms are often used to limit the available space. Because space can be divided into smaller spaces, these spaces can be recursively cut into smaller spaces. The idea of a priority heuristic algorithm is to recursively partition the space. According to the degree of fit of the rectangular block to be cut, the highest scored rectangular block can be cut from this space, and the remaining space is further recursively cut by this method. There are five different ways in which a rectangular block can be cut in space, as shown in Fig. 1. It can be seen that Fig. 1a is the optimal cutting method because the rectangular block fits perfectly with the space. Fig. 1b and Fig. 1c are slightly worse because the rectangular block is cut and a complete space remains. However, the remaining space of Fig. 1d needs to be further divided, and it is necessary to decide whether to cut horizontally or vertically. Fig. 1e shows that the space is too small and cannot accommodate any rectangular blocks. In Fig. 1b and Fig. 1c, the priority depends on the size of the space, the remaining objects to be cut, and the cutting rule settings. For example, the rectangular blocks to be cut may be arranged in descending order of height, in which case Fig. 1b is better than Fig. 1c, because higher-height rectangular blocks may be preferentially selected for cutting. Similarly, if the rectangular blocks to be cut are arranged in descending order of width, Fig. 1c is better than Fig. 1b.

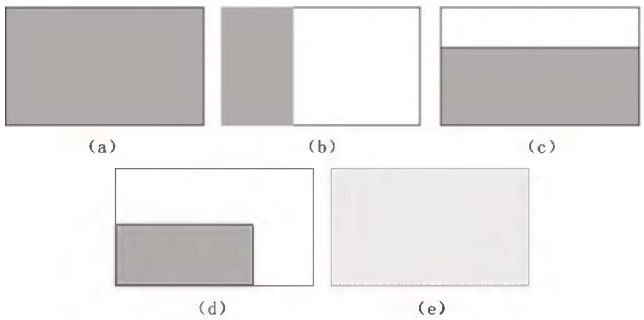

Figure 1. Cutting ways

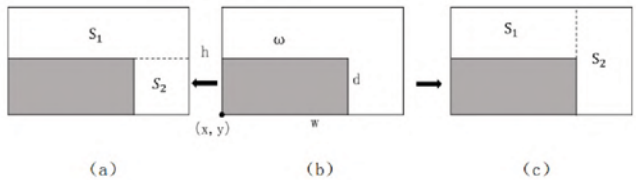

Figure 2. Crosscut and vertical cut

Assuming that the rectangular blocks are sorted by height, the priorities of the Fig. 1a, Fig. 1b, Fig. 1c, Fig. 1d and Fig. 1e are 1, 2, 3, 4, and 5, respectively, where 1 has the highest priority. The rectangle with the highest priority is cut first, and when it encounters the case of Fig. 1e, the cut is stopped because no rectangular blocks can match. For case Fig. 1b and Fig. 1c, the total amount of space has not changed. If we encounter two rectangular blocks have the same priority, select the one that is sorted first. For the Fig. 1d, the cutting strategy of the space is as follows: Let $minw$ and $minh$ respectively represent the minimum values of the width and height of the remaining rectangular blocks to be cut. According to Fig. 2b, if $w - \omega < minw$, the space S is divided into two spaces S_1 and S_2 as shown in Fig. 2a, meaning that the space S_2 is wasted because it cannot accommodate the width of any remaining rectangular blocks. Similarly, if $h - d < minh$, the space S is divided into two spaces S_1 and S_2 as shown in Fig. 2c, meaning that the space S_1 is wasted. If the above two conditions are not satisfied, then decide whether $\omega < minw$, if so, select the division method shown in Fig. 2a, because the minimum width of the remaining rectangular blocks is larger than the object ω, S_1 in Fig. 2c is obviously not wide enough for cutting. We just assumed that the rectangular blocks are sorted by height. In

Authorized licensed use limited to: Universita degli Studi di Bologna. Downloaded on March 25,2026 at 14:29:37 UTC from IEEE Xplore. Restrictions apply.

fact, the division of space depends largely on the ordering of the rectangular blocks. When the rectangular blocks are sorted by width, we tend to choose the dividing method of Fig. 2c, because objects with large width are easier to select, and $\omega <$ minw hardly occurs.

Algorithm 1 describes the recursive cutting algorithm for space S, where D indicates whether the direction limit is considered, 0 indicates that the rectangular block direction is fixed and cannot be rotated, and 1 indicates that it can be rotated by 90°. For rectangular block r, if $D = 0$, ω and d represent the width and height of the rectangle, respectively, otherwise ω indicates the short side and d means the long side. Priority 1 to 4 correspond to a to d of Fig. 1, respectively. We start cutting from the smallest of the remaining available space. Because if you start from a large space, a small rectangular block may occupy the space of a large rectangular block, and then the small space can no longer accommodate an object with a large area.

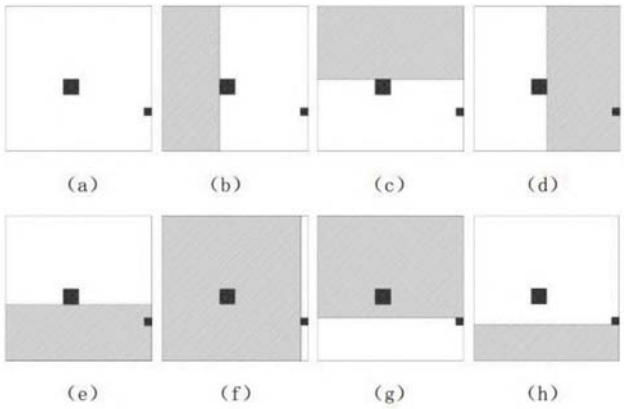

Figure 3. Schematic diagram of CATS algorithm cutting line

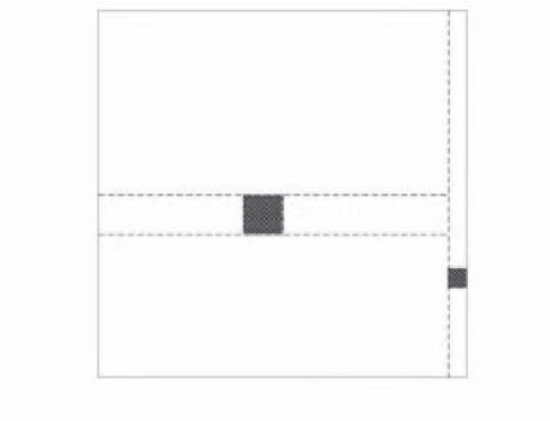

Figure 4. Schematic diagram of CATS algorithm space cutting

B. CATS algorithm

First, cut all the defective parts to get some blank space. Since the distribution of defects is irregular, a defect has 4 sides if it is cut along the edge of the defect. Assuming there are n defects, and only one knife can be cut at a time, there are a total of $(4n)!$ cutting methods. Some defects may overlap each other, or the edges are on the same horizontal or vertical line, in which case the cut selection is less than $4n$. Guillotine constraint divides a space into two. The number of cutting lines in each space is less than half of the original space, so the total cutting method is less than $(4n)!$. There are two defects (black) in Fig. 3a, and one of them is close to the edge of the rectangular block. There are currently 7 cutting lines that can be selected, as shown in Fig. 3b - h. The cutting line divides the original space into gray parts and the rest. These parts are divided into three cases, the first is a complete defect; the second is a space without defects, which can be added to the set of available space; the third is the space with defects, in this state then continue cutting in the

Algorithm 1: Priority Heuristic Algorithm

Input: $\boldsymbol{x, y, w, h}$

Output: Cutting result

```
if no free space left or all rectangular blocks are cut then
  return;
else
  foreach rectangular block i to be cut do
    determine the priority of i according to D;
  end
  select the rectangular block r with the highest
    priority from the block to be cut;
  record direction j and priority of r;
  if priority < 5 then
    set the length and width of r according to j of r;
    switch priority do
      case 2 do
        RecursiveCutting(x + ω, y, w − ω, h);
        break;
      case 3 do
        RecursiveCutting(x, y + d, w, h − d);
        break;
      case 4 do
        if w − ω < minw then
          cut into S1 and S2 as Fig. 2;
          discard space S2;
          RecursiveCutting(x, y + d, w, h − d);
        else if h − d < minh then
          cut into S1 and S2 as Fig. 2c;
          discard space S1;
          RecursiveCutting(x + ω, y, w − ω, h);
        else if ω < minw
          cut into S1 and S2 as Fig. 2a;
        else
          cut into S1 and S2 as Fig. 2c;
          RecursiveCutting(a smaller space);
          RecursiveCutting(a larger space);
        end
        break;
      otherwise do
        break;
      end
    end
  end
end
```


Authorized licensed use limited to: Universita degli Studi di Bologna. Downloaded on March 25,2026 at 14:29:37 UTC from IEEE Xplore. Restrictions apply.

same way until all the defects have been cut down, and the space without defects is added to the set of available space. Choosing which of the 7 cutting lines to cut down determines the space shape and cutting line of the next step. We propose two ways. The first way is to select one cutting line with the largest gray space area for each selection. Obviously, Fig. 3f It is the best choice; the other way is to randomly select a cutting line at each step, set the number of cycles, and optimize the results. For both methods, we conducted a comparison test in the experimental results. Fig. 4 shows one of cutting sequences in which the white portion is the resulting six spaces. Algorithm 2 describes the implementation of the CATS algorithm.

Algorithm 2: CATS Algorithm

Input: Rectangular block to be cut

Output: Cutting result

initialize a collection of spaces without defects $S \leftarrow \emptyset$;
initialize the defect set $D \leftarrow \emptyset$;
initialize L as a rectangular block with defects;
while $L \neq \emptyset$ **do**
  select a space l with defects from L;
  determine cutting lines according to the defect in l;
  select a cutting line to split l into p_1 and p_2;
  foreach p in $\{p_1, p_2\}$ **do**;
    if p is a collection of spaces without defects **then**
      $S \leftarrow S \cup \{p\}$;
    else if p is a complete defect **then**
      $D \leftarrow D \cup \{p\}$;
    else
      $L \leftarrow L \cup \{p\}$;
    end
  end
end
sort the space in set S;
foreach s in S **do**
  RecursiveCutting(s);
end

C. CIP algorithm

We arrange all the rectangular blocks in descending order of height, then start cutting along the lower left corner. Fig. 5 shows the first rectangular block (gray) cut smoothly from a rectangular block with two defects (black), and the remaining space is divided into two blocks S_1 and S_2 by the dotted line. Fig. 6 shows three scenarios when cutting the space S_2 in Fig. 5. The first case is that the rectangular block exceeds the length limit of the space, as shown in Fig. 5a; the second case is that the rectangular block overlaps with one or more defects, as shown in Fig. 5b; the third can be cut smoothly, as shown in Fig. 5c. When the rectangular block can be rotated, it is necessary to try the length and width of the two directions when determining the priority of the rectangular block to be cut, and take the one with the better priority. When the two rectangular blocks have the same priority, select the one with the higher height. If we traverse all the rectangular blocks, but no available block is found, the rectangular blocks that do not meet the conditions are not all because the width or height is too long, but because of the existence of the defect, as shown in Fig. 6b. In this case, the defect and the space in the lower left corner must be cut together. However, there are two cutting ways as is shown in Fig. 6a and Fig. 6b. In order to determine the cross-cutting or vertical-cutting to obtain a better spatial segmentation to facilitate the subsequent cutting process, we compare the size of the space S_1 and the space S_2 in Fig. 6a and Fig. 6b, and select a smaller cutting mode. Fig. 6b is significantly better than Fig. 6a.

Algorithm 3 describes the concrete implementation of the CIP algorithm. First calculate the priority of all remaining rectangular blocks to be cut in the current space, and then select a rectangular block with the highest priority. cw and ch respectively indicate the width and length of the portion to be cut, and the second and third lines indicate that a rectangular block that can be cut is found. As shown in Fig. 6c, cw and ch are respectively assigned the width and length of the rectangular block. Lines 5-7 indicate that if a defect blocking cut is encountered, then a recently encountered defect d_i is selected, assigning cw and ch. The reason for taking the minimum value is to consider the space containing defects and the intersection of space and defects. Line 8 indicates that if there is no blockage of the defect and no rectangular block is found for cutting, it exits. From the 11th line to the end, we judge whether it is cross-cut or vertical. Taking the defect blocking in Fig. 7 as an example, compare the size of the space S_1 in the Fig. 7a and the space S_2 in the Fig. 7b. If the space S_1 in Fig. 7a is smaller than the space S_2 in the Fig. 7b, 11-18 lines are executed. Then cut the defects as shown in Fig. 7a, otherwise perform lines 19-24 and cut the defects as shown in Fig. 7b.

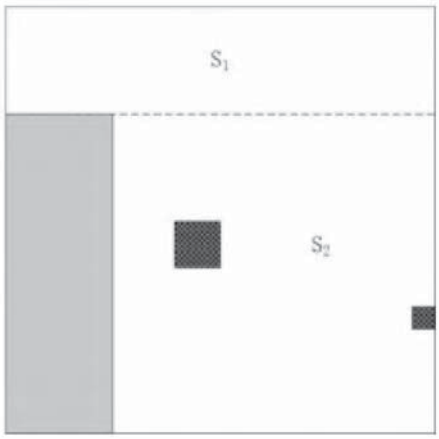

Figure 5. CIP algorithm cutting diagram

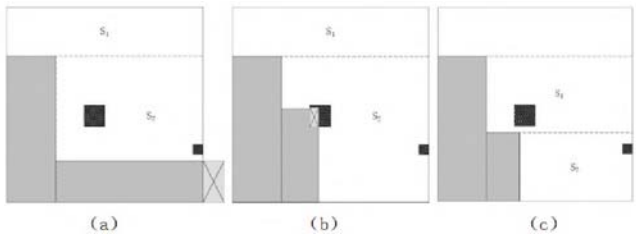

Figure 6. CIP algorithm cutting diagram

Authorized licensed use limited to: Universita degli Studi di Bologna. Downloaded on March 25,2026 at 14:29:37 UTC from IEEE Xplore. Restrictions apply.

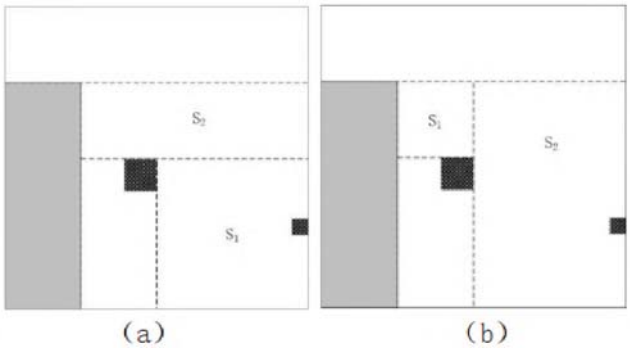

Figure 7. CIP algorithm cross-cutting or vertical-cutting diagram

Algorithm 3: CIP Algorithm

Input: $\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{W}, \boldsymbol{H}$

Output: Cutting result

```
calculate the priority of all remaining rectangular blocks;
select the highest priority rectangle r_j with priority;
if priority < 5 then
    cw ← ω^{r_j}, ch ← h^{r_j};
else if defect d_i blocks the cutting process then
    cw ← min(x_2^{d_i} − x, W);
    cw ← min(y_2^{d_i} − y, H);
else
    return;
end
if (W − cw) * ch < cw * (H − ch) then
    if cw < W then
        RecursiveCutting(x + cw, y, W − cw, ch);
    else
        if ch < H then
            RecursiveCutting(x, y + ch, W, H − ch);
        end
    end
else if cw < W
    RecursiveCutting(x + cw, y, W − cw, H);
else
    if ch < H then
        RecursiveCutting(x, y + ch, cw, H − ch);
    end
end
```

V. RESULT

The experiment used the newly generated 20 sets of data sets to verify the performance of the two algorithms. The evaluation index is the single rectangular block occupancy rate. It should be noted that the rectangular blocks in the 17th data set can be completely loaded, and the usage rate of the rectangular block is 62.61%.

For the CATS and CIP algorithms, we compare the results of the two algorithms under different constraints. Restrictions include:

- Rectangular block rotation and guillotine constraint: For both algorithms, we tested the results of the rectangular block orientation (OG) and the rectangular block direction at 90° rotation (RG).
- How to order rectangular blocks: We tested two rectangular block sorting strategies, sorted by height from large to small and sorted by area from largest to smallest. If sorted by height, in the case of the same priority, the rectangle with the higher height is preferentially selected. If the order is sorted by area, the rectangle with the larger area is preferred in the case of the same priority.
- Selection of cutting lines in CATS algorithm: In the CATS algorithm, when cutting a rectangular plate, it is necessary to determine the order in which the cutting lines are used. We tested two strategies, the remaining cut area maximum strategy and the random selection strategy, which are described in detail in Section IV-B.

Tables II and I show the experimental results of the CATS and CIP algorithms under the above constraints. We can see that the CIP algorithm performs better in the RG case and the rectangular block height priority. The CATS algorithm performs better in the OG case, when the cutting line is random and the block height is prioritized.

Table III compares the results of these three experiments. Among them, the tree search algorithm has two sets of experimental results, RG and OG, and the occupancy rate of both blocks is set to 1. The comparison results show that the tree search algorithm has an absolute advantage, but it should be noted here that the tree search algorithm does not meet guillotine constraint, that is, the cutting restrictions are less, and the space utilization is more flexible. We mainly compare the advantages and disadvantages of CATS algorithm and CIP algorithm. The CIP algorithm performs better in most cases and is much simpler from a data structure perspective. The CATS algorithm uses multiple random cuts, and the CIP algorithm has lower time complexity.

VI. CONCLUSIONS

In this paper, we have studied the two-dimensional cutting problem with defects. Based on the priority heuristic algorithm, we propose the CATS algorithm and the CIP algorithm to solve the problem by avoiding defects and excavating defects. We built our own test data sets and compared the performance of these algorithms. Compared with the same literature [27] as the research in this paper, the data set of this paper is larger and more universal.

ACKNOWLEDGMENT

This work is partially supported by the National Nature Science Foundation of China (Grant no. 61672439).

Authorized licensed use limited to: Universita degli Studi di Bologna. Downloaded on March 25,2026 at 14:29:37 UTC from IEEE Xplore. Restrictions apply.

TABLE I. EXPERIMENT RESULTS OF CATS ALGORITHM

Data	RG				OG			
	Random		Specific		Random		Specific	
	Height	Area	Height	Area	Height	Area	Height	Area
1	77.83	76.55	63.49	65.83	81.45	**84.13**	66.54	67.08
2	81.75	74.04	41.91	65.60	**91.82**	90.38	80.37	85.96
3	89.18	90.50	81.46	83.54	**95.13**	93.52	88.67	88.77
4	66.10	71.88	66.10	71.88	**97.89**	95.65	**97.89**	95.65
5	83.97	51.97	80.69	52.22	**95.12**	94.90	87.46	91.33
6	85.33	57.70	77.89	49.14	94.19	**94.47**	88.53	88.29
7	62.73	80.56	41.23	61.81	**85.37**	85.06	72.90	73.53
8	77.06	73.46	71.67	68.07	**82.83**	82.56	66.47	71.78
9	74.17	77.08	74.17	77.08	88.25	**89.86**	88.25	**89.86**
10	94.33	93.61	93.29	91.95	**98.95**	97.84	95.84	96.31
11	86.46	87.17	67.56	84.73	**94.96**	94.02	90.37	90.87
12	77.25	68.48	69.37	57.97	**94.23**	92.52	90.13	88.29
13	86.80	80.35	58.07	57.07	**94.68**	93.11	85.81	77.58
14	79.88	79.02	66.74	73.75	**84.35**	82.16	74.17	76.72
15	76.48	71.25	62.86	70.62	79.91	**84.95**	74.63	76.99
16	87.96	81.98	68.17	69.87	94.09	**94.44**	91.36	89.03
17	**62.61**	**62.61**	**62.61**	**62.61**	**62.61**	**62.61**	**62.61**	**62.61**
18	88.84	88.84	83.41	82.94	**93.74**	92.67	91.87	90.07
19	91.76	54.82	91.76	54.82	**96.02**	94.24	96.02	94.24
20	81.71	85.47	81.43	80.38	91.29	**92.32**	87.38	87.39

TABLE II. EXPERIMENT RESULTS OF CIP ALGORITHM

Data	OG		RG	
	Area	Height	Area	Height
1	72.32	84.21	84.00	**89.09**
2	88.39	91.49	87.17	**92.04**
3	89.52	**96.17**	93.43	93.06
4	95.15	97.37	**97.80**	96.11
5	96.99	94.27	**97.31**	96.32
6	92.58	91.46	92.65	**94.62**
7	81.99	84.48	**85.11**	80.57
8	87.19	76.98	**87.69**	87.09
9	**90.17**	85.80	86.77	88.85
10	95.95	96.03	97.60	**98.13**
11	93.29	89.51	92.95	**93.58**
12	92.12	**93.40**	88.63	93.20
13	**92.97**	91.00	86.89	90.24
14	81.88	65.69	**84.81**	83.16
15	87.35	83.84	76.85	**91.24**
16	87.73	92.37	80.92	**93.37**
17	**62.61**	**62.61**	**62.61**	**62.61**
18	90.13	88.11	90.80	**91.04**
19	93.21	95.79	92.95	**96.94**
20	92.57	93.06	90.24	**94.32**

TABLE III. COMPARISON OF THREE ALGORITHMS

Data	Tree Search RF	Tree Search OF	CIP	CATS
1	97.21	96.57	**89.09**	81.45
2	98.51	97.59	**92.04**	91.82
3	97.75	97.47	93.06	**95.13**
4	99.99	99.86	96.11	**97.89**
5	99.60	99.28	**96.32**	95.12
6	99.02	98.44	**94.62**	94.19
7	97.91	94.13	80.57	**85.37**
8	96.06	93.84	**87.09**	82.83
9	99.96	99.82	**88.85**	88.25
10	99.65	99.60	98.13	**98.95**
11	98.96	98.55	93.58	**94.96**
12	98.69	97.92	93.20	**94.23**
13	98.96	98.68	90.24	**94.68**
14	96.05	95.00	83.16	**84.35**
15	96.94	96.57	**91.24**	79.91
16	98.75	97.91	93.37	**94.09**
17	62.61	62.61	**62.61**	**62.61**
18	99.22	98.65	91.04	**93.74**
19	99.38	98.95	**96.94**	96.02
20	98.61	97.69	**94.32**	91.29

Authorized licensed use limited to: Universita degli Studi di Bologna. Downloaded on March 25,2026 at 14:29:37 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] C. H. Dagli and P. Poshyanonda, "New approaches to nesting rectangularpatterns,"Journal of Intelligent Manufacturing, vol. 8, no. 3, pp. 177–190, 1997.

[2] S. C. Leung, Y. Lin, and D. Zhang, "Extended local search algorithmbased on nonlinear programming for two-dimensional irregular strippacking problem," Computers & Operations Research, vol. 39, no. 3,pp. 678–686, 2012.

[3] D. Zhang, J. Chen, Y. Liu, and H. Chen, "Discrete no-fit polygon, a simple structure for the 2-d irregular packing problem," Journal ofSoftware, vol. 20, no. 6, pp. 1511–1520, 2009.

[4] J. Beasley, "An exact two-dimensional non-guillotine cutting tree searchprocedure,"Operations Research, vol. 33, no. 1, pp. 49–64, 1985.

[5] S. Martello and D. Vigo, "Exact solution of the two-dimensional finitebin packing problem," Management science, vol. 44, no. 3, pp. 388–399,1998.

[6] B. S. Baker, E. G. Coffman, Jr, and R. L. Rivest, "Orthogonal packingsin two dimensions,"SIAM Journal on computing, vol. 9, no. 4, pp. 846–855, 1980.

[7] B. Chazelle, "The bottomn-left bin-packing heuristic: An efficient implementation,"IEEE Transactions on Computers, no. 8, pp. 697–707,1983.

[8] A. Lodi, S. Martello, and D. Vigo, "Heuristic and metaheuristic approaches for a class of two-dimensional bin packing problems,"INFORMS Journal on Computing, vol. 11, no. 4, pp. 345–357, 1999.

[9] K. A. Dowsland, "Some experiments with simulated annealing techniques for packing problems,"European Journal of Operational Research, vol. 68, no. 3, pp. 389–399, 1993.

[10] S. Hong, D. Zhang, H. C. Lau, X. Zeng, and Y.-W. Si, "A hybrid heuristicalgorithm for the 2d variable-sized bin packing problem,"EuropeanJournal of Operational Research, vol. 238, no. 1, pp. 95–103, 2014.

[11] S. Jakobs, "On genetic algorithms for the packing of polygons,"European journal of operational research, vol. 88, no. 1, pp. 165–181,1996.

[12] D. Liu and H. Teng, "An improved bl-algorithm for genetic algorithm ofthe orthogonal packing of rectangles,"European Journal of OperationalResearch, vol. 112, no. 2, pp. 413–420, 1999.

[13] J. A. Bennell, L. S. Lee, and C. N. Potts, "A genetic algorithm fortwo-dimensional bin packing with due dates,"International Journal ofProduction Economics, vol. 145, no. 2, pp. 547–560, 2013.

[14] A. Lodi, S. Martello, and D. Vigo, "Tspack: a unified tabu searchcode for multi-dimensional bin packing problems,"Annals of OperationsResearch, vol. 131, no. 1-4, pp. 203–213, 2004.

[15] Y. Wang and L. Chen, "Two-dimensional residual-space-maximizedpacking,"Expert Systems with Applications, vol. 42, no. 7, pp. 3297–3305, 2015.

[16] L. Wei, W.-C. Oon, W. Zhu, and A. Lim, "A skyline heuristic for the 2d rectangular packing and strip packing problems,"European Journalof Operational Research, vol. 215, no. 2, pp. 337–346, 2011.

[17] S. C. Leung and D. Zhang, "A fast layer-based heuristic for non-guillotine strip packing,"Expert Systems with Applications, vol. 38,no. 10, pp. 13 G. Scheithauer and J. Terno, "Guillotine cutting of defective boards,"optimization, vol. 19, no. 1, pp. 111–121, 1988. 032–13 042, 2011.

[18] D. Zhang, L. Wei, S. C. Leung, and Q. Chen, "A binary search heuristicalgorithm based on randomized local search for the rectangular strip-packing problem,"INFORMS Journal on Computing, vol. 25, no. 2, pp.332–345, 2013.

[19] Ö. B. Aşık and E. Özcan, "Bidirectional best-fit heuristic for orthogonal rectangular strip packing". Annals of Operations Research, vol. 172, no. 1, p. 405, 2009.

[20] E. Özcan, Z. Kai, and J. H. Drake, "Bidirectional best-fit heuristicconsidering compound placement for two dimensional orthogonal rect-angular strip packing,"Expert Systems with Applications, vol. 40, no. 10,pp. 4035–4043, 2013.

[21] S. G. Hahn, "On the optimal cutting of defective sheets,"OperationsResearch, vol. 16, no. 6, pp. 1100–1114, 1968.

[22] G. Scheithauer and J. Terno, "Guillotine cutting of defective boards,"optimization, vol. 19, no. 1, pp. 111–121, 1988.

[23] E. De Gelder and A. Wagelmans, "The two-dimensional cutting stockproblem within the roller blind production process,"Statistica Neer-landica, vol. 63, no. 4, pp. 474–489, 2009.

[24] C. Carnieri, G. A. Mendoza, and W. G. Luppold, "Optimal cuttingof dimension parts from lumber with a defect: a heuristic solutionprocedure,"Forest Products Journal, vol. 43, no. 9, p. 66, 1993.

[25] V. Neidlein, A. C. Vianna, M. N. Arenales, and G. Wäscher, "Two-dimensional guillotineable-layout cutting problems with a single defect-an and/or-graph approach," inOperations Research Proceedings 2008.Springer, 2009, pp. 85–90.

[26] M. Afsharian, A. Niknejad, and G. Wäscher, "A heuristic, dynamicprogramming-based approach for a two-dimensional cutting problemwith defects,"OR spectrum, vol. 36, no. 4, pp. 971–999, 2014.

[27] L. Wenshu, M. Dan, and W. Jinzhuo, "Study on cutting stock optimization for decayed wood board based on genetic algorithm,"The OpenAutomation and Control Systems Journal, vol. 7, no. 1, 2015.

[28] R. Ghodsi and F. Sassani*, "Real-time optimum sequencing of woodcutting process,"International Journal of Production Research, vol. 43,no. 6, pp. 1127–1141, 2005.

[29] D. T. Aksu and B. Durak, "A dynamic programming algorithm forthe online cutting problem with defects and quality grades,"IFAC-PapersOnLine, vol. 49, no. 12, pp. 17–22, 2016.

[30] W. Zhu and A. Lim, "A new iterative-doubling greedy–lookaheadalgorithm for the single container loading problem," European Journalof Operational Research, vol. 222, no. 3, pp. 408–417, 2012.

[31] D. Zhang, L. Shi, S. C. Leung, and T. Wu,"A priority heuristic for theguillotine rectangular packing problem,"Information Processing Letters,vol. 116, no. 1, pp. 15–21, 2016.

Authorized licensed use limited to: Universita degli Studi di Bologna. Downloaded on March 25,2026 at 14:29:37 UTC from IEEE Xplore. Restrictions apply.